# ML Unit 1

# Machine Learning and Applications - Unit 1 Complete Notes

---

# Part 1: Foundations

# 1. Introduction to Machine Learning

## What is Machine Learning?

**Definition:** Machine Learning is the science of getting computers to learn without being explicitly programmed.

**Key Characteristics:**

- Ability to **learn and improve from experience**
- New capability for computers to **mimic human decision-making**
- Focus on pattern recognition and prediction rather than explicit programming

## Course Outcomes

Students completing this course should be able to:

1. Use effective machine learning techniques for various applications
2. Identify characteristics of datasets and choose appropriate learning models
3. Implement practical machine learning algorithms
4. Evaluate learning methods and relate them to particular problems
5. Design practical machine learning systems and apply ML to projects

---

# 2. Why Machine Learning?

## Success Factors

Machine learning has become successful due to three main factors:

**1. Abundant Data**

- Massive amounts of data generated daily
- Diverse data sources (sensors, social media, transactions, etc.)
- Improved data collection and storage capabilities

**2. Availability of Models and Algorithms**

- Extensive research and development
- Open-source libraries and frameworks
- Proven algorithms for various tasks

**3. Fast and Cheap Computing**

- Cloud computing infrastructure
- GPU acceleration for parallel processing
- Cost-effective computational resources

### Problem-Solving Approach

**Why use ML instead of traditional programming?**

1. **Hard to write explicit programs** for complex tasks like face recognition
2. We don't fully understand how our brain solves certain problems
3. Even if we understood, the program might be **horrendously complicated**

**ML Solution:**

- Collect **lots of examples** that specify correct output for given input
- ML algorithm takes these examples and **produces a program**
- The learned program may contain **millions of numbers** (parameters)
- If done correctly, the program **generalizes** to new, unseen cases

---

## 3. Applications of Machine Learning

### Pattern Recognition

- **Facial recognition** - identity and expression detection
- **Handwritten/spoken word** recognition
- **Medical imaging** - disease diagnosis and analysis

### Pattern Generation

- Generating images or motion sequences
- Creative applications (art, music)

### Anomaly Detection

- **Credit card fraud** - unusual transaction patterns
- **Industrial monitoring** - nuclear power plant sensors

- **Automotive diagnostics** - unusual engine sounds

### Prediction Tasks

- **Financial forecasting** - stock prices, currency exchange rates
- **Weather prediction**
- **Sales forecasting**

### Web-Based Applications

- **Spam filtering** - adaptive to evolving spam techniques
- **Fraud detection** - enemy adapts, system must adapt
- **Recommendation systems** - handling noisy data
- **Information retrieval** - finding similar documents/images
- **Data visualization** - revealing patterns using techniques like Latent Semantic Analysis

---

## 4. Types of Learning

### 1. Supervised Learning

**Definition:** Learn to predict output when given an input vector

**Characteristics:**

- Training data includes both **inputs (features)** and **outputs (labels)**
- Model learns the **mapping function** from input to output
- Used for **classification** and **regression** tasks

**Example:** Predicting house prices based on features (size, location, etc.)

### 2. Unsupervised Learning

**Definition:** Create internal representation of input (e.g., form clusters, extract features)

**Characteristics:**

- **No labeled outputs** provided
- Model discovers **hidden patterns** or structure
- Challenging to evaluate - "How do we know if representation is good?"
- **New frontier** of ML - most big datasets lack labels

**Example 1: Customer Segmentation**

- Shopping mall groups customers based on annual income and spending score

- No predefined labels like "High spender" or "Low spender"
- Algorithm: K-Means Clustering
- Results:
  - Cluster 1: Low Income – Low Spending
  - Cluster 2: High Income – High Spending

**Example 2: Image Segmentation**

- Dividing an image into meaningful regions
- Each pixel treated as a data point
- No ground-truth segmentation given
- Model discovers structure based on pixel similarity

## 3. Reinforcement Learning

**Definition:** Learn action to maximize payoff/reward through trial and error

**Characteristics:**

- Agent learns through **interaction** with environment
- Receives **rewards or penalties**
- **Limited information** in payoff signal
- Payoff is often **delayed** (temporal credit assignment problem)

**Applications:**

- Game playing (Chess, Go)
- Robotics
- Autonomous vehicles

---

# 5. Classification vs Regression

## Classification

**Definition:** Predict **discrete class labels**

**Characteristics:**

- Output is **categorical** (finite set of classes)
- Can be **binary** (2 classes) or **multi-class** (>2 classes)

**Examples:**

- **Medical diagnosis** - # classes = # diseases

- **Spam detection** - # classes = 2 (spam vs. not spam)
- **Image classification** - object recognition

### Regression

**Definition:** Predict **continuous numerical values**

**Characteristics:**

- Output is a **real number**
- Establishes **relationship** between variables

**Univariate Regression:**

- Single independent variable
- Example: Height vs. Weight prediction

**Multivariate Regression:**

- Multiple independent variables
- Example: Housing dataset with multiple features

**Real-World Example:** Marketing company tracks:

- Advertisement spending: [100, 150, 200, 250, 300]
- Corresponding sales: [500, 750, 1000, 1250, 1500]
- **Goal:** Predict sales for $200 advertisement in 2026

**Other Examples:**

- House price prediction
- Weather forecasting
- Electricity consumption
- Age estimation
- Traffic flow prediction
- Heart rate/blood pressure prediction

---

# Part 2: Data Management

## 6. Data Splitting Strategy

**Critical Concept:** Split data into training, validation, and test sets for proper model development

### Why Split Data?

Using a single dataset can cause:

- **Overfitting:** Model memorizes data but fails on new inputs
- **Biased performance estimates**
- **Poor generalization**

Splitting ensures:

- Fair evaluation
- Better generalization to unseen data
- Reliable performance metrics

## 1. Training Set (~60-70% of data)

**Purpose:** Learning the patterns

**What happens:**

- Model fits parameters (weights, biases, tree splits, etc.)
- Errors are minimized using optimization algorithms
- Model learns relationship between inputs (features) and outputs (labels)

**Why needed:**

- Without training data, the model cannot learn anything
- Foundation for all subsequent model development

## 2. Validation Set (~15-20% of data)

**Purpose:** Tuning and model selection

**What happens:**

- Adjust **hyperparameters** (learning rate, number of layers, k in k-NN)
- Decide **when to stop training** (early stopping)
- **Compare** different models or configurations

**Why needed:**

- Prevents **overfitting** to training data
- Ensures decisions are **not biased** by test set
- Allows iterative model improvement

## 3. Test Set (~15-20% of data)

**Purpose:** Final unbiased evaluation

**What happens:**

- Provides **unbiased estimate** of real-world performance
- **No learning or tuning** done using test data
- Used **only once** to evaluate final model

**Why needed:**

- If test data influences training, results become **misleading and overly optimistic**
- Ensures honest performance reporting

## Important Considerations

**Multiple Runs Needed:**

1. Training and validation sets might contain misleading factors (noise, outliers)
2. Learning method may depend on random factors (e.g., initial weights in MLP)
3. Run many times and average results to overcome randomness

**Using scikit-learn:**

```
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split

# create dataset
X, y = make_blobs(n_samples=1000)

# split into train test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# Output: (670, 2) (330, 2) (670,) (330,)
```

---

# 7. Practical Student Pass/Fail Example

## Problem Statement

Predict whether a student will **Pass (1)** or **Fail (0)** based on **hours studied**

## Dataset

- 10 labeled data points total
- Feature: Hours studied
- Label: Pass/Fail

## Training Phase (6 data points)

**Model learns:**

- Low study hours → Fail
- Higher study hours → Pass
- **Decision boundary** learned ≈ 4.5 hours
- Model fitted **only using training data**

### Validation Phase (2 data points)

**Purpose:** Test different model settings

- Test threshold = 4 hours vs. threshold = 5 hours
- Choose model with **better validation accuracy**
- Helps **avoid overfitting** and select best configuration

### Testing Phase (2 data points)

**Final Evaluation:**

- Accuracy on test set = 100%
- Test data **never used** during training or tuning
- Provides **unbiased performance** estimate

---

# Part 3: Learning Theory

## 8. Learning Class From Examples

### Example: Family Car Classification

**Problem Setup:**

- Class C = Family car
- Features: Price ($x_1$) and Engine Power ($x_2$)
- Positive examples (+): Cars within acceptable range
- Negative examples (-): Cars outside acceptable range

**Example Ranges:**

- $x_1'$ = 4 lakhs to $x_1$ = 14 lakhs (Price)
- $x_2'$ = 2 hp to $x_2$ = 8 hp (Engine Power)

### Formal Representation

**Each car represented by:**

$$x = [x_1, x_2]^T$$

```
where x₁ = Price, x₂ = Engine Power
```

**Labeling:**

```
r = { 1  if x is positive example
    { 0  if x is negative example
```

**Training Set:**

$$X = \{(x^t, r^t)\}^{t=N}_{(t=1)}$$

## Hypothesis Class

**Let H be the hypothesis class:**

$$H: (p_1 \leq \text{price} \leq p_2) \text{ AND } (e_1 \leq \text{Engine Power} \leq e_2)$$

Where:

- $p_1$ = Lower limit of price
- $p_2$ = Upper limit of price
- $e_1$ = Lower limit of engine power
- $e_2$ = Upper limit of engine power

**Example:**

```
(4 lakh ≤ price ≤ 14 lakh) AND (2 ≤ Engine Power ≤ 8)
```

## Learning Algorithm

The learning algorithm finds hypothesis $h \in H$ specified by $(p_1^h, p_2^h, e_1^h, e_2^h)$

**Goal:** Find $h \in H$ that is as similar as possible to C

**Hypothesis Function:**

```
h(x) = { 1  if h classifies x as positive
       { 0  if h classifies x as negative
```

## Empirical Error

**Error of hypothesis h given training set X:**

$$E(h|X) = \Sigma^{t=N}_{(t=1)} \mathbb{1}(h(x^t) \neq r^t)$$

Where:

$$1(h(x^t) \neq r^t) = \begin{cases} 1 & \text{if } h(x^t) \neq r^t \\ 0 & \text{if } h(x^t) = r^t \end{cases}$$

**Goal:** Select h such that E = 0

## Version Space

**Most Specific Hypothesis (S):**

- Tightest rectangle covering all positive examples
- Example: `8 lakh ≤ price ≤ 10 lakh AND (4 ≤ Engine Power ≤ 6)`

**Most General Hypothesis (G):**

- Loosest rectangle covering all positive examples
- Example: `4 lakh ≤ price ≤ 14 lakh AND (2 ≤ Engine Power ≤ 8)`

**Version Space:**

- Any h $\in$ H between S and G
- All hypotheses consistent with training set (zero error)

**True Class C (unknown):**

```
C: 4 lakh ≤ price ≤ 13 lakh AND (2.5 ≤ Engine Power ≤ 7.5)
```

## Errors in Classification

**False Negative:**

- Point where C = 1 but h = 0
- Example: Price = 12.75 lakhs, Engine Power = 4
- Actual class says positive, but hypothesis says negative

**False Positive:**

- Point where C = 0 but h = 1
- Example: Price = 10 lakhs, Engine Power = 7.9
- Actual class says negative, but hypothesis says positive

## S-Set and G-Set

In reality, there may be multiple specific and general hypotheses:

**S-Set:**

- Several $S_i$ make up S-Set
- Every member is consistent with all instances
- No consistent hypothesis more specific exists

**G-Set:**

- Several $G_j$ make up G-Set
- Every member is consistent with all instances
- No consistent hypothesis more general exists

**Version Space:** All hypotheses between S-Set and G-Set

**Candidate Elimination Algorithm:**

- Incrementally updates S-Set and G-Set
- Processes training instances one by one
- Assumption: Training set large enough for unique S and G

### Margin and Doubt

**Margin:**

- Choose hypothesis with **largest margin** for best separation
- Shaded instances define margin
- Other instances can be removed without affecting h
- Maximizes separation between classes

**Doubt (Reject Option):**

- Hypothesis falls between S and G
- Cannot label due to lack of data
- System rejects instance and defers to human expert
- Example: h between S and G with intermediate boundaries

---

## 9. Vapnik-Chervonenkis (VC) Dimension

### Definition

**VC Dimension:**

- Measure of the capacity (complexity) of a hypothesis class
- Maximum number of points that can be **shattered** by H

### What is "Shattering"?

**Definition:** A set of points is **shattered** by a hypothesis class if all possible labelings of those points can be correctly classified by some hypothesis in the class.

**For n points:**

- Total possible labelings = $2^n$
- If model can represent all labelings → points are shattered

## Why VC Dimension is Important

VC dimension helps to:

1. **Measure model complexity**
2. **Understand overfitting vs generalization**
3. **Provide theoretical guarantees** on learning
4. **Choose appropriate model** for given data

## Examples

### 1. Threshold Classifier on a Line

**Model:**

```
h(x) = { 1  if x ≥ c
       { 0  otherwise
```

**VC Dimension = 1**

- 1 point → can be labeled as 0 or 1
- 2 points → cannot realize all labelings

### 2. Linear Classifier in 2D

**VC Dimension = 3**

- 3 non-collinear points → All $2^3 = 8$ labelings possible
- 4 points → Some labelings cannot be separated by a line
- Example: XOR problem cannot be solved by linear classifier

### 3. Linear Classifier in d Dimensions

**General Formula:**

```
VC Dimension = d + 1
```

**Examples:**

- 2D → VC = 3
- 3D → VC = 4

### 4. Axis-Aligned Rectangle in 2D

**VC Dimension = 4**

- Can shatter 4 points
- Cannot shatter 5 points

## Intuitive Understanding

**Low VC dimension:**

- Simple model
- May underfit
- Fewer training samples needed

**High VC dimension:**

- Complex model
- Risk of overfitting
- More training samples needed

## VC Dimension and Generalization

**Generalization error depends on:**

1. Training error
2. VC dimension
3. Number of training samples

**Key Insight:**

- Higher VC dimension requires **more training data** to generalize well
- Trade-off between model complexity and data requirements

## Limitations

1. **Hard to compute** for complex models
2. **Mostly theoretical** - difficult to apply in practice
3. **Loose bounds** in practice - often too conservative

---

# 10. PAC Learning (Probably Approximately Correct)

# Basic Concept

**PAC Learning Framework:** A theoretical framework that defines when a learning algorithm can be considered successful.

**PAC stands for:**

- **Probably** → with high probability
- **Approximately** → small error allowed
- **Correct** → close to the true concept

# The Problem

**Goal:** Find number of examples N such that with probability (1-δ), hypothesis h has error at most ε

**Mathematical Formulation:**

```
P(CΔh ≤ ε) = 1 − δ
```

Where:

- CΔh = Region of difference between C and h
- ε = Error bound (accuracy parameter)
- δ = Confidence parameter

# Derivation for Rectangle Hypothesis

**Setup:**

- h = S (tightest possible rectangle)
- Error region = sum of four rectangular strips

**Step-by-Step:**

1. Probability that each positive example falls in strip = ε/4
2. Probability that example misses strip = 1 - ε/4
3. Probability N examples miss one strip = $(1 - \varepsilon/4)^N$
4. Probability N examples miss all four strips = $4(1 - \varepsilon/4)^N$

**Using approximation:**

```
(1 − ε/4)^N ≈ e^(−εN/4)
```

**Setting bound:**

```
4e^(-εN/4) ≤ δ
e^(-εN/4) ≤ δ/4
-εN/4 ≤ ln(δ/4)
N ≥ (4/ε)ln(4/δ)
```

## Interpretation

**Sample Complexity:**

```
N ≥ (4/ε)ln(4/δ)
```

**Insights:**

1. As δ decreases (higher confidence) → N increases
2. As ε decreases (lower error) → N increases
3. Polynomial relationship with 1/ε and log(1/δ)

## Formal PAC Definition

A hypothesis class H is **PAC-learnable** if for:

- Error bound ε > 0
- Confidence bound δ > 0

There exists a learning algorithm A such that:

```
P(error(h) ≤ ε) ≥ 1 - δ
```

using a **polynomial number** of samples.

## Numerical Example

**Assume:**

- ε = 0.05 (5% error allowed)
- δ = 0.01 (99% confidence)

**Meaning:** With 99% probability, the learned model's error will be ≤ 5% on unseen data.

## Role of VC Dimension

**Sample complexity depends on VC dimension (d):**

```
m = O((1/ε)(d·log(1/ε) + log(1/δ)))
```

Where:

- d = VC dimension
- Higher VC → more samples needed

## Examples of PAC-Learnable Models

**1. Threshold Functions on a Line**

- VC dimension = 1
- Requires few samples
- PAC-learnable

**2. Linear Classifiers**

- Finite VC dimension
- PAC-learnable

## Why PAC Learning is Important

1. **Theoretical guarantee** of learning
2. **Connects** model complexity and generalization
3. **Explains** why some models need more data
4. **Foundation** of learning theory

---

# 11. Noise in Machine Learning

## What is Noise?

**Definition:** Noise is an unwanted anomaly in the data. Due to noise, the class may be difficult to learn and zero error may be impossible with a simple hypothesis class.

## Types of Noise

### 1. Input Noise

**Imprecision in recording input attributes**

- Shifts data points in input space
- Measurement errors
- Sensor inaccuracies

### 2. Label Noise (Teacher's Noise)

**Errors in labeling data points**

- Relabels positive examples as negative (and vice versa)

- Human annotation errors
- Ambiguous boundary cases

### 3. Hidden/Latent Variables

**Additional attributes not taken into account**

- Attributes that affect the label
- May be unobservable
- Effect modeled as random component
- Included in noise term

## Effect of Noise

**Without noise:**

- Simple hypothesis (e.g., rectangle) works well
- Clean separation possible

**With noise:**

- Need complicated hypothesis to separate instances
- Risk of overfitting to noise
- Zero error may be impossible

## Why Use Simple Models Despite Noise?

**Occam's Razor Principle:** Given comparable empirical error, a simple model generalizes better than a complex model.

**Reasons for simplicity:**

1. **Easy to use**
   - Simple to check (e.g., point inside/outside rectangle)
   - Fast prediction for future instances
2. **Simple to train**
   - Fewer parameters to estimate
   - Easier optimization
   - Less variance, more bias
   - Optimal model minimizes both bias and variance
3. **Simple to explain**
   - Rectangle = defining intervals on attributes
   - Interpretable results
   - Can extract information from raw data
4. **Better generalization**

- If actual class is simple (e.g., rectangle)
- Resists overfitting to noise
- More robust to variations

---

# 12. Learning Multiple Classes

## Problem Setup

**K Classes:** $C_1, C_2, ..., C_k$

**Example:**

- Class $C_1$ = Family Car
- Class $C_2$ = Luxury Car
- Class $C_3$ = Sports Car

## Representation

**Training Set:**

$$X = \{(x^t, r^t)\}^{t=N}_{(t=1)}$$

Where r is K-dimensional:

$$r^t_i = \begin{cases} 1 & \text{if } x^t \in C_i \\ 0 & \text{if } x^t \in C_j \text{ for } j \neq i \end{cases}$$

## K Hypotheses

Learn K hypotheses $\{h_1, h_2, ..., h_k\}$ such that:

$$h_i(x^t) = \begin{cases} 1 & \text{if } x^t \in C_i \\ 0 & \text{if } x^t \in C_j \text{ for } j \neq i \end{cases}$$

**Strategy:**

- For each class $C_i$, positive instances = class $C_i$
- Negative instances = all other classes
- Learn boundary separating $C_i$ from rest

## Total Empirical Error

**Formula:**

$$E(\{h_i\}^k_{i=1}|X) = \Sigma^{t=N}_{(t=1)} \Sigma^k_{i=1} 1(h_i(x^t) \neq r^t_i)$$

Where:

- K = Number of classes
- N = Number of instances

## Decision Cases

For a given x:

**Case 1: Ideal**

- Only one $h_i(x) = 1$
- Clear classification

**Case 2: Ambiguous**

- Two or more $h_i(x) = 1$
- Cannot choose unique class
- Need tie-breaking strategy

**Case 3: Doubt/Reject**

- No $h_i(x) = 1$
- Insufficient confidence
- Classifier rejects instance
- Defer to human expert

---

# Part 4: Model Evaluation

# 13. Model Evaluation & Performance Metrics

## Why Evaluate?

**Two Key Questions:**

1. **How can we assess expected error** of a learning algorithm?
   - Given a trained classifier on a dataset
   - How confident are we about real-life error rate?
2. **How can we compare** two learning algorithms?
   - For a given problem
   - Which is better: MLP or k-NN?

## Important Principles

**Training error is NOT a deciding factor:**

- Error on training set < Error on testing set
- More complex model with more parameters → lower training error
- But may not generalize well

**Always use validation set:**

- Not just one run - multiple runs needed
- Average results to overcome randomness
- Training and validation sets may contain noise/outliers

**Multiple runs needed because:**

1. Small datasets may have misleading factors
2. Learning method depends on random factors
3. Example: MLP with backprop converges to local minimum
4. Initial weights affect final weights → different error rates

**Process:**

```
1 training = 1 learner + 1 validation error
Multiple trainings → Multiple learners → Distribution of errors
Evaluation based on this distribution
```

---

# 14. Confusion Matrix

## Basic Concepts

**For binary classification:**

**True Positive (TP):**

- Predicted value matches actual value
- Actual = Positive, Predicted = Positive

**True Negative (TN):**

- Predicted value matches actual value
- Actual = Negative, Predicted = Negative

**False Positive (FP) - Type 1 Error:**

- Predicted value was falsely predicted
- Actual = Negative, Predicted = Positive
- Also called "False Alarm"

**False Negative (FN) - Type 2 Error:**

- Predicted value was falsely predicted
- Actual = Positive, Predicted = Negative
- Also called "Miss"

## Example

**Dataset: 1000 data points**

Confusion Matrix:

```
                Predicted
                Pos    Neg
Actual  Pos     560    50
        Neg     60     330
```

**Values:**

- TP = 560 (positive correctly classified)
- TN = 330 (negative correctly classified)
- FP = 60 (negative incorrectly classified as positive)
- FN = 50 (positive incorrectly classified as negative)

**Assessment:** Decent classifier with relatively large TP and TN values

## Performance Metrics

**Accuracy:**

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
         = (560 + 330) / 1000
         = 0.89 or 89%
```

**True Positive Rate (Sensitivity, Recall, Hit Rate):**

```
TPR = TP / (TP + FN)
    = 560 / (560 + 50)
    = 0.918 or 91.8%
```

Measures proportion of actual positives correctly identified.

**False Positive Rate (False Alarm Rate):**

```
FPR = FP / (FP + TN)
    = 60 / (60 + 330)
    = 0.154 or 15.4%
```

Measures proportion of actual negatives incorrectly identified as positive.

**Specificity:**

```
Specificity = TN / (TN + FP) = 1 - FPR
            = 330 / (330 + 60)
            = 0.846 or 84.6%
```

How well we detect negatives.

### Multi-Class Confusion Matrix

**For K > 2 classes:**

- K × K matrix
- Entry (i, j) = number of instances belonging to $C_i$ but assigned to $C_j$
- **Ideally:** All off-diagonals should be 0 (no misclassification)
- Allows pinpointing what types of misclassification occur

---

## 15. ROC Curves and AUC

### Receiver Operating Characteristics (ROC)

**Concept:**

- For different threshold values θ
- Get pairs of (TPR, FPR) values
- Plot these pairs to create ROC curve

**Decision Rule:**

```
Choose positive if P(positive|x) / P(negative|x) > θ
```

### ROC Curve Analysis

**Perfect Classifier:**

- TPR = 1, FPR = 0

- Point at upper-left corner

**Better Classifier:**

- ROC curve closer to upper-left corner
- Higher TPR for same FPR

**Comparing Two Classifiers:**

- One is better if its ROC curve is above the other
- If curves intersect: different classifiers better under different loss conditions

### Area Under Curve (AUC)

**Purpose:** Reduce ROC curve to single number for comparison

**Interpretation:**

- AUC = 1: Perfect classifier
- AUC = 0.5: Random classifier
- Higher AUC = Better classifier

---

## 16. Precision, Recall, and F1-Score

### Information Retrieval Context

**Scenario:**

- Database of records
- Make query (e.g., keywords)
- System returns records (two-class classifier)

**Outcomes:**

- **True Positives:** Retrieved AND relevant
- **False Positives:** Retrieved but NOT relevant
- **False Negatives:** NOT retrieved but relevant
- **True Negatives:** NOT retrieved and NOT relevant

### Precision

**Definition:**

```
Precision = (Retrieved AND Relevant) / Total Retrieved
          = TP / (TP + FP)
```

**Meaning:**

- What proportion of retrieved records are relevant?
- If precision = 1: All retrieved may be relevant
- But there may be relevant records not retrieved

## Recall

**Definition:**

```
Recall = (Retrieved AND Relevant) / Total Relevant
       = TP / (TP + FN)
```

**Meaning:**

- What proportion of relevant records are retrieved?
- Same as Sensitivity and TPR
- If recall = 1: All relevant may be retrieved
- But there may be irrelevant records retrieved

## F1-Score

**Definition:**

```
F1-Score = 2 × (Precision × Recall) / (Precision + Recall)
         = 2TP / (2TP + FP + FN)
```

**Meaning:**

- Harmonic mean of Precision and Recall
- Balances both metrics
- Good for imbalanced datasets

## Relationships

**Sensitivity = TPR = Recall**

```
All three measure: TP / (TP + FN)
```

**Specificity:**

```
Specificity = TN / (TN + FP) = 1 - FPR
```

## Trade-offs

**High Precision, Low Recall:**

- Conservative system
- Returns few results, but mostly relevant

**Low Precision, High Recall:**

- Liberal system
- Returns many results, but includes irrelevant ones

**F1-Score:**

- Optimizes balance
- Useful when need both precision and recall

---

# Part 5: Regression

# 17. Parametric Methods

## What is a Parametric Method?

**Definition:** An approach that assumes a **fixed form (structure)** for the model and summarizes data using a **finite set of parameters**.

**Key Characteristics:**

1. Assumes specific form for the model
2. Uses finite set of parameters
3. Once parameters learned, **dataset can be discarded**
4. Model fully defined by parameters

## Example: Linear Regression

**Problem:** Predict sales based on advertisement

**Hypothesis:**

$$\text{Sales} = w_0 + w_1 \times \text{Advertisement}$$

**Parameters:**

- $w_0$: Intercept
- $w_1$: Slope

**Parametric Nature:**

- Once $w_0$ and $w_1$ are learned
- Labeled dataset can be discarded
- Model completely specified by two numbers

---

# 18. Regression Analysis

## Basic Concept

**In Classification:**

- Output is Boolean

**In Regression:**

- Output is a **numeric function**

## Problem Setup

**Training Set:**

$$X = \{(x^t, r^t)\}_{(t=1)}^{t=N}$$

**Goal:** Find function f(x) such that $r^t = f(x^t)$

## Types of Problems

**1. Interpolation (No Noise):**

- Find function passing through all points
- Task: fit function to data

**2. Polynomial Interpolation:**

- Given N points
- Find (N-1) degree polynomial
- Can predict for any x

**3. Extrapolation:**

- Prediction when x is outside range of $x^t$ in training set
- Extends beyond observed data

**4. Time-Series Prediction:**

- Have data up to present

- Predict value for future
- Special case of extrapolation

## Regression with Noise

**Model:**

$$r^t = f(x^t) + \varepsilon$$

Where:

- f(x): Unknown function to approximate
- ε: Random noise

**Noise Sources:**

- Extra hidden variables not observed
- Can write as: $r^t = f^*(x^t, z^t)$
- $z^t$: Hidden variables

## Error Functions

**Empirical Error (Mean Squared Error):**

$$E(g|X) = (1/N) \sum_{(t=1)}^{t=N} [r^t - g(x^t)]^2$$

Where:

- $g(x^t)$: Output by our model
- $r^t$: Actual output
- Goal: Minimize this error

---

# 19. Linear and Polynomial Regression

## Linear Regression (Single Input)

**Model:**

$$g(x|w_0, w_1) = w_0 + w_1 x$$

**Parameters:** $w_0$ (intercept) and $w_1$ (slope)

**Optimization:**

- Take partial derivatives of E with respect to $w_0$ and $w_1$
- Set equal to 0
- Solve for the two unknowns

**Solution (Least Squares):**

$$w_1 = \Sigma(x^t - \bar{x})(r^t - \bar{r}) / \Sigma(x^t - \bar{x})^2$$
$$w_0 = \bar{r} - w_1\bar{x}$$

Where:

- $\bar{x}$: Mean of x values
- $\bar{r}$: Mean of r values

**Vector Form:**

$$w = (X^TX)^{-1}X^Tr$$

## Polynomial Regression

**When to Use:** If linear model too simple (large approximation error)

**Quadratic Model:**

$$g(x|w_0, w_1, w_2) = w_0 + w_1x + w_2x^2$$

**General Polynomial (Order k):**

$$g(x|w) = w_0 + w_1x + w_2x^2 + ... + w_kx^k$$

**Parameters:** $w_0, w_1, w_2, ..., w_k$

**Matrix Form:** Still solvable using:

$$w = (X^TX)^{-1}X^Tr$$

Where X contains powers of x

## Multivariate Linear Regression

**Problem:** Sales depends on multiple factors:

- Advertisement ($x_1$)
- Fuel price ($x_2$)
- Interest rate ($x_3$)

**Model:**

$$\text{Sales} = w_0 + w_1 \times \text{Advertisement} + w_2 \times \text{Interest\_Rate} + w_3 \times \text{Fuel\_Price}$$

**Parameters:** $w_0$, $w_1$, $w_2$, $w_3$

**Dimension of Problem:**

- Number of features (d)
- Model order
- Sample size needed

**Key Insight:** As model order or number of features increases, need larger dataset to determine parameters.

## Goodness of Fit Measures

**1. Relative Square Error (RSE):**

$$\text{ERSE} = \Sigma[r^t - g(x^t)]^2 \; / \; \Sigma[r^t - \bar{r}]^2$$

**Interpretation:**

- ERSE = 1: Prediction as good as predicting average
- ERSE = 0: Perfect fit

**2. Coefficient of Determination ($R^2$):**

$$R^2 = 1 - \text{ERSE}$$

**Interpretation:**

- $R^2$ close to 1: Good fit
- $R^2 = 0$: Model no better than mean
- $R^2 < 0$: Model worse than mean

**For regression to be useful:** Require $R^2$ close to 1

## Probabilistic Interpretation

**Assumption:** Noise ε is Gaussian with:

- Mean = 0
- Variance = $\sigma^2$

**Model:**

$$r = g(x|\theta) + \varepsilon$$

**Log Likelihood:**

$$L = \log p(r|x, \theta) = -N/2 \log(2\pi\sigma^2) - (1/2\sigma^2)\Sigma[r^t - g(x^t|\theta)]^2$$

**Maximum Likelihood Estimation:**

- Maximizing log likelihood
- Equivalent to minimizing squared error
- Parameters that minimize MSE are **Least Squares Estimates**

---

# Part 6: Model Selection

# 20. Model Selection and Generalization

## The Ill-Posed Problem

**Example: Boolean Function**

- Number of inputs = d (e.g., $x_1$, $x_2$)
- Training instances = $2^d$ (e.g., 00, 01, 10, 11)
- Number of possible outputs = $2^{(2^d)}$

**For d=2:**

- 4 possible inputs
- 16 possible functions ($h_1$, $h_2$, ..., $h_{16}$)

**Problem:**

- Each training example removes inconsistent hypotheses
- Example: ($x_1$=0, $x_2$=1) → output=0 removes 8 hypotheses
- To end with unique hypothesis, need $2^d$ training examples

**With Small Training Set (N < $2^d$):**

- After N examples, remain $2^{(2^{d-N})}$ possible functions
- **Ill-posed problem:** Data not sufficient for unique solution
- Example: If d=2, N=3, then $2^{(2^{2-3})}$ = 2 functions remain

## Solution: Inductive Bias

**Definition:** The set of assumptions we make to have learning possible.

**Examples of Inductive Bias:**

1. **Family Car Example:**
   - Assumption 1: Shape is a rectangle
   - Assumption 2: Choose rectangle with largest margin
2. **Linear Regression:**
   - Assumption: Function is linear
3. **Polynomial Regression:**
   - Assumption: Function is polynomial of specific order

**Key Insight:** Learning is **not possible** without inductive bias.

---

# 21. Inductive Bias

## What is Inductive Bias?

**Formal Definition:** The set of assumptions that the learner uses to predict outputs for inputs it has not encountered.

**Purpose:** Makes learning possible by restricting hypothesis space.

## Types of Inductive Bias

**1. Language Bias:**

- Restricts hypothesis space H
- Example: "Consider only polynomials up to degree k"
- Example: "Use only axis-aligned rectangles"

**2. Search Bias:**

- Preference for certain hypotheses over others
- Example: "Choose simplest hypothesis"
- Example: "Prefer hypothesis with largest margin"

## Examples in Different Contexts

**Classification:**

- Rectangle shape (language bias)
- Largest margin (search bias)
- Axis-aligned vs. rotated rectangles

**Regression:**

- Linear function (language bias)
- Polynomial of order k (language bias)
- Least squares solution (search bias)

**Neural Networks:**

- Network architecture (language bias)
- Initialization strategy (search bias)
- Regularization (search bias)

### Choosing the Right Bias

This is the problem of **model selection:**

- Choosing between possible hypothesis classes H
- Trade-off between:
  - Simplicity (fewer parameters)
  - Flexibility (capturing complex patterns)

---

## 22. Underfitting and Overfitting

### Generalization

**Definition:** How well a model trained on the training set predicts the right output for new instances.

**Goal:** Match complexity of hypothesis class H with complexity of underlying function.

### Underfitting

**Definition:** Hypothesis class H is **less complex** than the true function.

**Characteristics:**

- Model too simple
- **High bias, low variance**
- Poor fit to training data
- Poor performance on test data

**Example:**

- Trying to fit a line to data from a cubic polynomial
- Linear model for clearly non-linear relationship

**Effect:**

- As we increase complexity, training error **decreases**
- Model can't capture true pattern

## Overfitting

**Definition:** Hypothesis class H is **more complex** than needed.

**Characteristics:**

- Model too complex
- **Low bias, high variance**
- Excellent fit to training data (including noise)
- Poor performance on new data

**Example:**

- Fitting 6th order polynomial to noisy data from 3rd order polynomial
- Model learns noise instead of signal

**Effect:**

- Training error very low
- Test error high
- Model doesn't generalize

## The Triple Trade-off

**Three factors interact:**

1. **Complexity** of hypothesis class (capacity)
2. **Amount** of training data
3. **Generalization** error on new examples

**Key Insights:**

- More training data helps, but only up to a point
- Right complexity depends on amount of data
- Need balance between all three factors

## Visual Understanding

**Underfitting:**

```
True function: y = x³
Model: y = mx + b (straight line)
Result: Can't capture curvature
```

**Good Fit:**

```
True function: y = x³
Model: y = ax³ + bx² + cx + d
Result: Captures pattern well
```

**Overfitting:**

```
True function: y = x³ (with noise)
Model: y = polynomial of degree 10
Result: Fits every point including noise
```

---

# 23. Bias-Variance Tradeoff

## Mathematical Framework

**Expected squared error at point x:**

```
E[(r - g(x))²] = σ² + E[(g(x) - E[r|x])²]
```

**Decomposition:**

```
E[(r - g(x))²] = Noise + (Bias² + Variance)
```

Where:

- **$\sigma^2$**: Variance of noise (irreducible error)
- **Bias²**: How much g(x) deviates from true function
- **Variance**: How much g(x) fluctuates with different datasets

## Components Explained

**1. Noise ($\sigma^2$):**

- Irreducible error
- Present in data
- Cannot be eliminated by better model

**2. Bias:**

```
Bias[g(x)] = E[g(x)] - E[r|x]
```

- Measures systematic error
- How much estimator is wrong on average
- Disregards effect of varying samples

**3. Variance:**

```
Var[g(x)] = E[(g(x) - E[g(x)])²]
```

- Measures sensitivity to training data
- How much g(x) fluctuates around expected value
- Varies as sample varies

## The Dilemma

**To Decrease Bias:**

- Model should be **flexible**
- At risk of having **high variance**
- Can fit complex patterns
- But sensitive to data variations

**To Decrease Variance:**

- Keep model **simple**
- At risk of having **high bias**
- Stable predictions
- But may miss important patterns

**Optimal Model:**

- Best trade-off between bias and variance
- Minimizes total error
- Depends on problem and data

## Polynomial Order Example

**As polynomial order increases:**

**Low Order (e.g., degree 1):**

- **High bias** - can't fit curvature
- **Low variance** - stable across datasets
- **Underfitting**

**Medium Order (e.g., degree 3):**

- **Medium bias** - reasonable fit
- **Medium variance** - some sensitivity
- **Good fit**

**High Order (e.g., degree 10):**

- **Low bias** - fits training data well
- **High variance** - very sensitive to data
- **Overfitting**

## Practical Implications

**1. Model Complexity:**

- Simple models: High bias, low variance
- Complex models: Low bias, high variance

**2. Training Data Size:**

- Small datasets: Prefer simple models (avoid variance)
- Large datasets: Can use complex models

**3. Noise Level:**

- High noise: Prefer simple models
- Low noise: Can use complex models

**4. Ensemble Methods:**

- Average many high-variance models
- Reduces variance while maintaining low bias
- Example: Random Forests

---

# 24. Cross-Validation

## Purpose

**Model Selection:** Choose optimal model complexity

- Too simple → underfitting
- Too complex → overfitting
- Need systematic way to find sweet spot

## Basic Approach

**Split data into two parts:**

1. **Training set** - train models
2. **Validation set** - test models

**Process:**

1. Train models of different complexities on training set
2. Evaluate each on validation set
3. Choose model with best validation performance

## Performance vs. Complexity

**Training Error:**

- Decreases as complexity increases
- Always goes down or stays same
- Not reliable for model selection

**Validation Error:**

- Decreases initially
- Reaches minimum at optimal complexity
- Increases beyond optimal point (overfitting)

**The "Elbow":**

- Point where validation error stops decreasing
- Indicates optimal complexity level
- Balance between bias and variance

## k-Fold Cross-Validation

**Procedure:**

1. Divide data into k equal parts (folds)
2. For each fold i:
    - Train on remaining k-1 folds
    - Test on fold i
3. Average performance across all k folds

**Advantages:**

- Uses all data for both training and testing
- More reliable estimate of performance
- Reduces variance in estimates

**Common choices:**

- k=5 (5-fold CV)
- k=10 (10-fold CV)
- k=N (Leave-One-Out CV)

## Other Model Selection Methods

**1. Regularization:**

- Add penalty for model complexity
- Examples: L1 (Lasso), L2 (Ridge)

**2. Akaike Information Criterion (AIC):**

- Balances fit quality and complexity
- Penalizes number of parameters

**3. Bayesian Information Criterion (BIC):**

- Similar to AIC
- Stronger penalty for complexity

**4. Structural Risk Minimization:**

- Theoretical framework
- Based on VC dimension

**5. Minimum Description Length:**

- Information-theoretic approach
- Balances model complexity and data fit

**6. Bayesian Model Selection:**

- Uses posterior probabilities
- Incorporates prior beliefs

---

# Part 7: Advanced Classification

# 25. Dimensions of Supervised ML

## Three Key Decisions

To build a good approximation $g(x|\theta)$ to desired output r, must make three decisions:

# 1. Model g(·|θ)

**Definition:**

- Defines the **hypothesis class H**
- g(·): The model structure
- θ: Parameters to be learned
- Particular θ value instantiates one hypothesis $h \in H$

**Examples:**

**Class Learning:**

- Model: Rectangle
- Parameters θ: Four coordinates ($p_1$, $p_2$, $e_1$, $e_2$)

**Linear Regression:**

- Model: Linear function g(x|m,c) = mx + c
- Parameters θ: Slope (m) and intercept (c)
- Example: y = 2x + 5 where m=2, c=5

**Decision Process:**

- Model (inductive bias) fixed by **system designer**
- Based on knowledge of application
- Hypothesis tuned by **learning algorithm**
- Using training set sampled from p(x,r)

# 2. Loss Function L(·)

**Purpose:** Compute difference between desired output $r^t$ and approximation $g(x^t|\theta)$

**Total Error:**

$$E(\theta|X) = \Sigma^{t=N}_{(t=1)} L(r^t, g(x^t|\theta))$$

**Examples:**

**Classification (0/1 Loss):**

$$L(r, g(x)) = \begin{cases} 0 & \text{if } r = g(x) \\ 1 & \text{if } r \neq g(x) \end{cases}$$

Checks for equality.

**Regression (Squared Error):**

```
L(r, g(x)) = [r − g(x)]²
```

Uses ordering information for distance.

## 3. Optimization Procedure

**Goal:** Find θ* that minimizes total error

**Formula:**

```
θ* = arg min_θ E(θ|X)
```

Where "arg min" returns the argument that minimizes.

**Considerations:**

**Analytical Solution:**

- Polynomial regression: Can solve analytically
- Closed-form solution exists

**Iterative Methods:**

- Other models need iterative optimization
- Complexity becomes important factor

**Local vs. Global Minima:**

- **Single minimum:** Globally optimal solution
- **Multiple minima:** Locally optimal solutions
- Affects optimization strategy

## Success Requirements

**For the approach to work well:**

1. **Sufficient Capacity:**
   - Hypothesis class H should be large enough
   - Must include (approximately) the unknown function
   - Generated the data in noisy form
2. **Enough Data:**
   - Sufficient training data to pinpoint correct hypothesis
   - From the hypothesis class
   - More complex models need more data
3. **Good Optimization:**

- Effective method to find correct hypothesis
- Given the training data
- Avoid local minima when possible

---

# 26. Parametric Classification

## Problem Setup

**Goal:** Classify based on continuous features using probabilistic approach

**Posterior Probability:**

$$P(C_i|x) = p(x|C_i)P(C_i) / p(x)$$

**Discriminant Function:**

$$g_i(x) = \log P(C_i|x) = \log p(x|C_i) + \log P(C_i) - \log p(x)$$

Since log p(x) same for all classes, can be ignored:

$$g_i(x) = \log p(x|C_i) + \log P(C_i)$$

## Gaussian Assumption

**Assume $p(x|C_i)$ is Gaussian:**

$$p(x|C_i) = (1/\sqrt{(2\pi\sigma_i^2)}) \exp(-(x-\mu_i)^2/(2\sigma_i^2))$$

**Discriminant becomes:**

$$g_i(x) = -(x-\mu_i)^2/(2\sigma_i^2) - \tfrac{1}{2}\log(2\pi\sigma_i^2) + \log P(C_i)$$

## Example: Car Sales

**Scenario:**

- Car company selling K different car types
- x = customer's annual income
- $P(C_i)$ = Proportion of customers buying car type i
- $\mu_i$ = Mean annual income of customers buying car i
- $\sigma_i^2$ = Variance of annual income for car i

# Parameter Estimation

**When $P(C_i)$, $\mu_i$, $\sigma_i^2$ unknown:**

**Prior Probability:**

$$\hat{P}(C_i) = N_i/N$$

Where $N_i$ = number of samples in class $C_i$

**Mean:**

$$\hat{\mu_i} = (1/N_i) \sum_{t \in C_i} x^t$$

**Variance:**

$$\hat{\sigma_i^2} = (1/N_i) \sum_{t \in C_i} (x^t - \hat{\mu_i})^2$$

# Special Cases

### 1. Equal Priors and Equal Variances

If $P(C_i)$ same for all classes and $\sigma_i^2 = \sigma^2$:

$$g_i(x) = -(x-\mu_i)^2/(2\sigma^2)$$

**Decision Rule:** Assign x to class with **nearest mean**

**Formula:**

$$\text{Choose } C_i \text{ if } |x - \mu_i| < |x - \mu_j| \text{ for all } j \neq i$$

### 2. Two Classes with Equal Variances

**Decision boundary:**

$$x* = (\mu_1 + \mu_2)/2$$

**Interpretation:**

- Single threshold
- Linear decision boundary
- Midpoint between class means

### 3. Two Classes with Unequal Variances

**Result:**

- **Two thresholds** possible
- Quadratic decision boundary
- Posteriors intersect at two points

---

# 27. Parameter Estimation

## Multivariate Data

**Setup:**

$$X = [x^1, x^2, ..., x^N]^T$$

Where:

- d columns represent d variables/features/attributes
- N rows represent N observations/examples/instances

**Example: Loan Application**

- N customers
- Features: age, annual income, assets

## Sample Statistics

**1. Mean Vector:**

$$\hat{\mu} = (1/N) \sum_{(t=1)}^{t=N} x^t$$

Each element is mean of one column.

**2. Variance:**

$$\sigma_i^2 = (1/N) \sum_{(t=1)}^{t=N} (x_i^t - \mu_i)^2$$

**3. Covariance:**

$$\sigma_{ij} = (1/N) \sum_{(t=1)}^{t=N} (x_i^t - \mu_i)(x_j^t - \mu_j)$$

With i=j: $\sigma_{ij} = \sigma_{ji} = \sigma_{ii} = \sigma_i^2$

**4. Covariance Matrix:**

$$\Sigma = \begin{bmatrix} \sigma_{11} & \sigma_{12} & \dots & \sigma_{1}{}^{d} \\ \sigma_{21} & \sigma_{22} & \dots & \sigma_{2}{}^{d} \\ \dots & \dots & \dots & \dots \\ \sigma^{d}{}_{1} & \sigma^{d}{}_{2} & \dots & \sigma^{dd} \end{bmatrix}$$

Properties:

- Diagonal elements: variances
- Off-diagonal elements: covariances
- Symmetric matrix
- Size: d×d

**Matrix notation:**

$$\Sigma = (1/N)\ \Sigma^{t}\ (x^{t} - \mu)(x^{t} - \mu)^{T}$$

**5. Correlation:**

$$\rho_{ij} = \sigma_{ij} / (\sigma_{i}\sigma_{j})$$

Normalized between -1 and +1.

**6. Correlation Matrix R:** Contains all correlation coefficients $\rho_{ij}$

---

# 28. Multivariate Normal Distribution

## Definition

**Univariate Normal:**

- Single variable
- Parameters: μ (mean), $\sigma^2$ (variance)

**Multivariate Normal:**

- d-dimensional vector x
- Parameters: μ (mean vector), Σ (covariance matrix)

**Notation:**

$$x \sim N(\mu, \Sigma)$$

## Probability Density Function

**General Form:**

$$p(x) = (1/((2\pi)^{(d/2)}|\Sigma|^{(1/2)})) \exp(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu))$$

## Mahalanobis Distance

**Definition:**

$$D^2 = (x - \mu)^T\Sigma^{-1}(x - \mu)$$

**Properties:**

- Accounts for correlations between variables
- Variables with larger variance receive less weight
- Defines d-dimensional hyper-ellipsoid centered at μ
- Shape and orientation defined by Σ

**Interpretation:**

- Small $D^2$ → sample close to mean
- Large $D^2$ → sample far from mean
- Accounts for covariance structure

## Bivariate Case (d=2)

**Mean vector:**

$$\mu = [\mu_1, \mu_2]^T$$

**Covariance matrix:**

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{12} & \sigma_2^2 \end{bmatrix}$$

**Joint density:**

$$p(x_1, x_2) = (1/(2\pi\sigma_1\sigma_2\sqrt{(1-\rho^2)})) \exp(-\frac{1}{2}[(z_1^2 - 2\rho z_1 z_2 + z_2^2)/(1-\rho^2)])$$

Where:

- $z_1 = (x_1 - \mu_1)/\sigma_1$ (standardized variables)
- $z_2 = (x_2 - \mu_2)/\sigma_2$
- $\rho = \sigma_{12}/(\sigma_1\sigma_2)$ (correlation)

**This is called z-normalization.**

## Special Cases

### 1. Independent Variables (Naive Case)

When components independent and identically distributed:

$$\Sigma = \text{diagonal matrix}$$
$$\Sigma^{-1} = \text{diagonal matrix}$$

**Joint density becomes:**

$$p(x) = p(x_1) \times p(x_2) \times \ldots \times p(x^d)$$

Product of individual univariate densities.

**Example with $\sigma^2 = 1$:**

$$p(x) = (1/(2\pi)^{(d/2)}) \exp(-\tfrac{1}{2}\Sigma_i x_i^2)$$

### 2. Projection Property

If W is d×k matrix with rank k<d:

$$y = W^T x$$

Then:

$$y \sim N(W^T\mu, W^T\Sigma W)$$

**Meaning:**

- Projecting d-dimensional normal to k-dimensional space
- Result is k-dimensional normal
- Preserves normality under linear transformation

---

# 29. Multivariate Classification

## Problem Setup

**Goal:** Predict car type based on customer data

**Components:**

- **Classes:** Different car types (i)
- **Features:** x = [age, income]$^T$
- **$\mu_i$:** Mean vector for class i
- **$\Sigma_i$:** Covariance matrix for class i

**Example:**

- $\mu_i$ = mean [age, income] of customers buying car i
- $\sigma_1^2$ = variance of age in class i
- $\sigma_2^2$ = variance of income in class i
- $\sigma_{12}$ = covariance of age and income in class i

## Discriminant Functions

**Conditional density:**

$$p(x|C_i) \sim N(\mu_i, \Sigma_i)$$

**Discriminant:**

$$g_i(x) = -\frac{1}{2}\log|\Sigma_i| - \frac{1}{2}(x-\mu_i)^T\Sigma_i^{-1}(x-\mu_i) + \log P(C_i)$$

**Expanded form:**

$$g_i(x) = w_i^T x + w_{i0}$$

Where:

$$w_i = \Sigma_i^{-1}\mu_i$$
$$w_{i0} = -\frac{1}{2}\mu_i^T\Sigma_i^{-1}\mu_i - \frac{1}{2}\log|\Sigma_i| + \log P(C_i)$$

## Parameter Requirements

**Number of parameters:**

- **Means:** K × d
- **Covariances:** K × d(d+1)/2
- **Total:** K × [d + d(d+1)/2]

**Challenges with small samples:**

- $\Sigma_i$ may be **singular** (non-invertible)

- $|\Sigma_i|$ may be very small
- Estimates unreliable

**Solutions:**

1. Decrease dimensionality
2. Use common covariance matrix

## Simplifications

**1. Common Covariance Matrix ($\Sigma_i = \Sigma$)**

**Parameters:**

- Means: K × d
- Shared covariance: d(d+1)/2
- Total: K×d + d(d+1)/2

**Result:**

- **Linear decision boundaries**
- More stable with limited data

**Discriminant:**

$$g_i(x) = w_i^T x + w_{i0}$$

Where:

$$w_i = \Sigma^{-1}\mu_i$$
$$w_{i0} = -\frac{1}{2}\mu_i^T\Sigma^{-1}\mu_i + \log P(C_i)$$

**With equal priors:** Assign to class with smallest Mahalanobis distance.

**2. Naive Bayes Classifier**

**Assumption:** Features are independent

$$\Sigma = \text{diagonal matrix}$$

**Simplification:**

$$p(x|C_i) = \prod_j p(x_j|C_i)$$

**Advantages:**

- Drastically reduces parameters
- Fast training and prediction
- Works well even when independence violated

**3. Nearest Mean Classifier**

**Assumptions:**

- Common covariance
- All variances equal
- Features independent
- Result: $\Sigma = \sigma^2 I$

**Distance:** Euclidean distance

**Discriminant:**

$$g_i(x) = -||x - \mu_i||^2/(2\sigma^2) + \log P(C_i)$$

**With equal priors:**

$$g_i(x) = -||x - \mu_i||^2$$

**Decision:** Assign to class with nearest mean

**4. Further Simplification**

If $\mu_i$ have similar norms:

- Can neglect norm terms
- Even simpler decision rule

## Variance Reduction

**Trade-off:**

- **Simplicity** (few parameters)
- **Generality** (capture complex patterns)

**Strategies:**

1. Diagonal covariance (independence)
2. Shared covariance (common $\Sigma$)
3. Equal variances (all $\sigma^2$ equal)
4. Combinations

**Effect:**

- Decreases parameters
- Introduces **bias**
- Reduces **variance**

## Regularized Discriminant Analysis

**Weighted combination:**

$$\Sigma_i(\alpha, \beta) = \alpha I + (1-\alpha)[\beta\Sigma + (1-\beta)\Sigma_i]$$

Where $\alpha, \beta \in [0,1]$

**Special cases:**

- **(α=0, β=0):** Quadratic classifier
- **(α=0, β=1):** Linear classifier
- **(α=1, β=0):** Nearest mean classifier

**Tuning:** Use validation set to select optimal α and β.

## Decision Boundary Types

**Linear Classifier:**

- Straight line/hyperplane
- Shared covariance
- Fast, interpretable

**Quadratic Classifier:**

- Curved boundary (ellipse, parabola, hyperbola)
- Class-specific covariances
- More flexible

**Nearest Mean:**

- Can be piecewise linear
- Depends on class means and priors
- Simplest classifier

---

# 30. Discrete Features Classification

## Problem Setup

**Discrete attributes:** Variables taking one of n different values

**Examples:**

- Color $\in$ {red, green, blue}
- Pixel $\in$ {0, 1}
- Word presence $\in$ {0, 1}

## Binary Features

**Model:**

$$p(x|C_i) = \prod_j p_{ij}^{x_j}(1-p_{ij})^{1-x_j}$$

Where:

- **$p_{ij}$:** Probability feature j=1 in class i
- **$x_j$:** Binary feature (0 or 1)

**Discriminant:**

$$g_i(x) = \Sigma_j[x_j \log p_{ij} + (1-x_j)\log(1-p_{ij})] + \log P(C_i)$$

**Note:** Linear in x

**Parameter estimation:**

$$\hat{p}_{ij} = (\#\text{ times feature } j=1 \text{ in class } i) / (\text{total samples in class } i)$$

## Application: Document Classification

**Bag of Words:**

1. Choose d words a priori (vocabulary)
2. Each document → binary vector
   - $x_j = 1$ if word j appears
   - $x_j = 0$ otherwise

**After training:**

- $p_{ij}$ estimates probability word j appears in document type i
- Classify new documents based on word presence

## Multinomial Features

**Feature $x_j \in \{v_1, v_2, \ldots, v_m\}$**

**Model:**

$$p(x|C_i) = \prod_j \prod_k p_{ijk}^{I(x_j=v_k)}$$

Where:

- **$p_{ijk}$:** $P(x_j=v_k \mid C_i)$
- **$I(x_j=v_k)$:** Indicator function

**Independence assumption:**

$$p(x|C_i) = \prod_j p(x_j|C_i)$$

**Discriminant:**

$$g_i(x) = \Sigma_j \log p(x_j|C_i) + \log P(C_i)$$

**Parameter estimation:**

$$\hat{p}_{ijk} = (\#\text{ samples in } C_i \text{ where } x_j=v_k) / (\#\text{ samples in } C_i)$$

---

# Practice Problems and Examples

## Problem 1: Linear Regression

**Dataset:**

| $x_1$ | $x_2$ | y |
|---|---|---|
| 1 | 1 | 6.1 |
| 2 | 1 | 7.8 |
| 3 | 1 | 10.1 |
| 1 | 2 | 9.05 |
| 2 | 2 | 10.8 |
| 3 | 2 | 12.87 |
| 1 | 3 | 13 |
| 2 | 3 | 13.5 |
| 3 | 3 | 16.01 |

**Test data:**

| $x_1$ | $x_2$ | |
|---|---|---|
| 4 | 1 | (y = ?) |

**Tasks:**

1. Develop linear regression model
2. Calculate training error
3. Calculate testing error
4. Predict y when $x_1$=4.2, $x_2$=1.9

## Problem 2: Model Selection

**Training data:**

```
x    y
1    5.1
2    10.8
3    21.1
1.5  7.65
2.5  15.8
3.5  26.5
1.8  9.5
2.2  12.5
```

**Test data:**

```
x    y
4.3  40.1
4    35.5
```

**Tasks:**

1. Fit linear model, find training error
2. Fit quadratic model, find training error
3. Fit 4th degree polynomial, find training error
4. Test all three on test data
5. Plot training vs testing error
6. Comment on underfitting/overfitting

## Problem 3: Bias-Variance Analysis

**Two classifiers:**

- Model A: Linear Classifier
- Model B: k-NN (K=5)

**Results:**

```
           Training Error  Test Error
Model A          15%           18%
Model B           3%           22%
```

**Tasks:**

1. Identify which has high bias/variance
2. Justify using bias-variance tradeoff
3. Suggest which to select

## Problem 4: Parametric Classification

**Given:**

- Two classes $\omega_1$, $\omega_2$
- Prior: $P(\omega_1) = 0.6$, $P(\omega_2) = 0.4$
- $\omega_1$: $N(\mu_1=5, \sigma_1^2=4)$
- $\omega_2$: $N(\mu_2=10, \sigma_2^2=9)$

**Tasks:**

1. Calculate discriminant functions
2. Find decision boundary
3. Classify x=7
4. Calculate error probabilities

## Problem 5: Document Classification

**Features:**

- $x_1$: Presence of word "discount"
- $x_2$: Presence of word "free"

**Training data:**

```
Class    x₁   x₂   Count
Spam     Yes  Yes    50
Spam     Yes  No     10
Spam     No   Yes    20
Spam     No   No      5
Ham      Yes  Yes     5
Ham      Yes  No     15
Ham      No   Yes    10
Ham      No   No     85
```

**Tasks:**

1. Estimate class conditional probabilities
2. Estimate prior probabilities
3. Classify document with x=(Yes, Yes)

---

# Summary and Key Takeaways

## Fundamental Concepts

### 1. Machine Learning Paradigm

- Learn from examples, not explicit programming
- Requires: data, model, optimization
- Generalizes to new instances

### 2. Types of Learning

- Supervised: Classification and regression
- Unsupervised: Clustering, dimensionality reduction
- Reinforcement: Sequential decision making

### 3. Data Management

- Split: Training/Validation/Test
- Multiple runs for reliability
- Cross-validation for model selection

## Learning Theory

### 4. Hypothesis Space

- Version space bounded by S and G
- VC dimension measures capacity
- PAC learning provides guarantees

### 5. Model Selection

- Inductive bias necessary for learning
- Underfitting vs overfitting
- Bias-variance tradeoff

## Classification

### 6. Parametric Classification

- Assumes probabilistic model

- Gaussian assumption common
- Maximum likelihood estimation

### 7. Simplifications

- Common covariance → linear boundaries
- Independent features → Naive Bayes
- Equal variances → nearest mean

## Regression

### 8. Linear Models

- Least squares estimation
- Closed-form solution
- $R^2$ for goodness of fit

### 9. Polynomial Models

- Higher order for complex patterns
- Risk of overfitting
- Cross-validation for selection

## Evaluation

### 10. Performance Metrics

- Confusion matrix fundamentals
- Precision, Recall, F1-Score
- ROC curves and AUC

### 11. Model Complexity

- Training vs validation error
- Optimal complexity at elbow
- Multiple validation methods

## Best Practices

### 12. Model Development

- Start simple, add complexity if needed
- Use validation for hyperparameter tuning
- Test set for final unbiased evaluation

### 13. Practical Considerations

- Missing values → imputation
- Discrete features → multinomial models
- Noise → prefer simple models

**14. Trade-offs**

- Bias vs Variance
- Complexity vs Interpretability
- Computation vs Accuracy

---

## Textbook Reference

**Source:** "Introduction to Machine Learning" by Ethem Alpaydin, 4th Edition, MIT Press, 2020

---

*End of Unit 1 Complete Notes*